Azure Service Bus – Creating a Topic Subscription Programmatically

1. Overview

A topic subscription can be created programmatically from a .NET application instead of manually creating it through the Azure Portal.

In this example, an existing topic:

```
FirstTopic
```

already contains:

```
S1
S2
```

We create a new subscription:

```
S3
```

from Visual Studio.

Result

```
FirstTopic
   │
   ├── S1
   ├── S2
   └── S3  ← Created from C#
```

The important new concept here is the **Service Bus Administration Client**.

---

2. ServiceBusAdministrationClient

To create or manage Service Bus entities such as:

- Queues
- Topics
- Subscriptions

we use:

```
ServiceBusAdministrationClient
```

This client is different from `ServiceBusClient`.

ServiceBusClient

Used for **messaging operations**:

```
Send messages
Receive messages
Complete messages
```

ServiceBusAdministrationClient

Used for **management/administration operations**:

```
Create queue
Create topic
Create subscription
Delete queue
Delete topic
Delete subscription
Check entity existence
```

Easy Way to Remember

```
ServiceBusClient
       ↓
Message Operations

ServiceBusAdministrationClient
```

```
       ↓
Management Operations
```

---

3. Creating the Administration Client

Create the administration client using the Service Bus connection string:

```
var adminClient =
    new ServiceBusAdministrationClient(connectionString);
```

The connection string provides access to the Service Bus namespace and its entities.

---

4. Creating a Subscription

To create a subscription, use:

```
await adminClient.CreateSubscriptionAsync(
    topicName,
    subscriptionName);
```

For example:

```
string topicName = "FirstTopic";
string subscriptionName = "S3";

await adminClient.CreateSubscriptionAsync(
    topicName,
    subscriptionName);
```

This creates:

```
FirstTopic
    |
    └── S3
```

---

5. Creating a Subscription Without a Filter

In this example, the subscription is created **without specifying a custom filter**.

The basic creation method is:

```
await adminClient.CreateSubscriptionAsync(
    topicName,
    subscriptionName);
```

This is different from creating a subscription with a custom SQL filter, which can be covered separately.

The important point is:

> If no custom filter is specified, the subscription is created with the default subscription rule behavior.

---

6. Complete Example

```
using Azure.Messaging.ServiceBus.Administration;

string connectionString = "<connection-string>";

string topicName = "FirstTopic";
string subscriptionName = "S3";

var adminClient =
    new ServiceBusAdministrationClient(connectionString);

await adminClient.CreateSubscriptionAsync(
    topicName,
    subscriptionName);

Console.WriteLine("Subscription created successfully.");
```

---

7. Execution Flow

The application performs these steps:

```
Step 1
Connection String
       ↓
Step 2
Create ServiceBusAdministrationClient
       ↓
Step 3
Specify Topic Name
       ↓
Step 4
Specify Subscription Name
       ↓
Step 5
CreateSubscriptionAsync()
       ↓
Step 6
Subscription created
```

For this example:

```
Topic = FirstTopic
Subscription = S3
```

---

8. Azure Portal Result

Before running the application:

```
FirstTopic
   |
   ├── S1
   └── S2
```

After running the application:

```
FirstTopic
   |
   ├── S1
   ├── S2
   └── S3
```

The new `S3` subscription is created programmatically from Visual Studio.

---

9. Important Difference: Messaging vs Administration

This is an important interview concept.

Client	Purpose
`ServiceBusClient`	Send and receive messages
`ServiceBusAdministrationClient`	Manage Service Bus entities

Example

To send a message:

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);
```

To create a subscription:

```
ServiceBusAdministrationClient adminClient =
    new ServiceBusAdministrationClient(connectionString);
```

Remember

```
ServiceBusClient
       ↓
"Work with messages"

ServiceBusAdministrationClient
       ↓
"Work with Service Bus resources"
```

---

10. Real-Time Use Case

Suppose an application needs to dynamically create subscriptions for different departments.

For example:

```
Orders Topic
      |
      ├── Payment
      ├── Inventory
      └── Shipping
```

Instead of manually creating every subscription in the Azure Portal, an administrative application can create them programmatically.

For example:

```
await adminClient.CreateSubscriptionAsync(
    "Orders",
    "Shipping");
```

This is useful for automation and infrastructure-management scenarios.

---

11. Important Interview Questions

Q1. Which client is used to create a Service Bus subscription?

Answer:

`ServiceBusAdministrationClient`.

```
var adminClient =
    new ServiceBusAdministrationClient(connectionString);
```

Q2. Which method is used to create a subscription?

Answer:

```
await adminClient.CreateSubscriptionAsync(
    topicName,
    subscriptionName);
```

Q3. What is the difference between ServiceBusClient and ServiceBusAdministrationClient?

Answer:

`ServiceBusClient` is used for message operations such as sending and receiving messages.

`ServiceBusAdministrationClient` is used for management operations such as creating, updating, and deleting Service Bus entities.

Q4. Can we create a subscription programmatically?

Answer:

Yes. We can use `ServiceBusAdministrationClient` and call:

```
CreateSubscriptionAsync()
```

---

12. Key Points to Remember

- Subscriptions can be created through code.
- Use `ServiceBusAdministrationClient` for Service Bus management operations.
- Provide the **topic name** and **subscription name**.
- Use `CreateSubscriptionAsync()` to create the subscription.
- A subscription can be created without specifying a custom filter.
- `ServiceBusClient` is primarily for messaging.
- `ServiceBusAdministrationClient` is for administration/management.

Quick Memory Trick

```
SEND / RECEIVE MESSAGE
        ↓
ServiceBusClient


CREATE / DELETE / MANAGE
        ↓
ServiceBusAdministrationClient
```

One-Line Interview Answer

We use ServiceBusAdministrationClient to programmatically manage Azure Service Bus entities, such as creating a subscription under an existing topic.